

GESTÃO DE TURMAS COM RECURSO A UMA LISTA ORDENADA PRÓPRIA

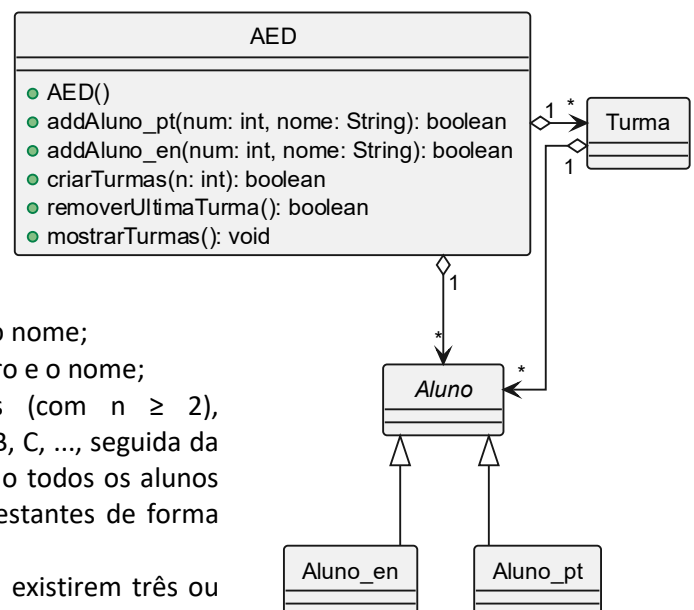
Pretende-se desenvolver uma aplicação para gestão de turmas de AED, iniciando-se pela implementação de uma estrutura de dados própria para o armazenamento das entidades envolvidas: a `LinkedListSorted`.

A lista é um ADT que permite armazenar elementos de forma sequencial, possibilitando o seu acesso através da posição que ocupam (indexação). Neste trabalho considera-se uma variante particular deste ADT: a lista ordenada. Nesta estrutura, os elementos são mantidos permanentemente ordenados de acordo com a sua ordem natural. Assim, sempre que um novo elemento for inserido, este deverá ser colocado na posição adequada, garantindo a preservação da ordenação.

Pretende-se, assim, a implementação de uma lista ordenada, designada por `LinkedListSorted`, cuja estrutura interna deverá basear-se numa das listas ligadas desenvolvidas nas aulas da unidade curricular. A lista deverá ser genérica e não permitir a existência de elementos duplicados.

Após a implementação desta estrutura, a mesma deverá ser utilizada na gestão de turmas de alunos. Cada aluno é caracterizado por um número e um nome, podendo ser de dois tipos: aluno nacional (`Aluno_pt`) ou aluno internacional (`Aluno_en`). As turmas deverão ser identificadas pelas letras iniciais do alfabeto (A, B, C, ...).

Para um melhor entendimento do problema, é apresentado o diagrama de classes UML (incompleto) correspondente à modelação pretendida, no qual apenas a classe `AED` se encontra totalmente especificada. Esta classe deverá centralizar toda a lógica do sistema, sendo responsável pela gestão dos alunos e das turmas. As funcionalidades da aplicação, refletidas nos métodos públicos desta classe, são as seguintes:



- Registo de alunos nacionais, dado o número e o nome;
- Registo de alunos internacionais, dado o número e o nome;
- Criação de turmas: criação de n turmas (com $n \geq 2$), automaticamente identificadas pelas letras A, B, C, ..., seguida da distribuição dos alunos já registados, colocando todos os alunos internacionais na turma A e distribuindo os restantes de forma equilibrada pelas restantes turmas;
- Remoção da última turma (apenas possível se existirem três ou mais turmas), voltando-se a redistribuir todos os alunos pelas restantes turmas;
- Apresentação de todas as turmas, juntamente com os seus alunos.

Por fim, deverá ser desenvolvido um método `main` que permita testar adequadamente a sua aplicação de gestão de turmas.

Trabalho a desenvolver

Implementar em Java uma solução para o problema enunciado que cumpra, nomeadamente, as especificações de implementação e de carácter geral a seguir descritas.

Considerações a ter em conta na implementação

1. A aplicação deverá ser desenvolvida no IDE NetBeans 17 ou posterior, o projeto deverá ter o nome `trab_pratico` e estar devidamente anonimizado. Se contiver qualquer elemento que permita identificar os autores, será **descontado 1 valor** à classificação do trabalho.

2. O diagrama de classes apresentado deverá ser integralmente respeitado, incluindo todos os membros da classe AED¹. Apenas é permitido acrescentar os atributos e métodos considerados adequados nas restantes classes.
3. Todas as coleções do problema deverão ser implementadas com a estrutura `LinkedList`.
4. A classe `LinkedList` a desenvolver deve implementar todo o comportamento definido na interface `SortedList` fornecida, e ter por base uma das listas ligadas desenvolvidas nas aulas da unidade curricular.
5. A interface `SortedList` é disponibilizada no ipb.virtual, subpasta <Recursos/avaliacao/[TrabPrat](#)> da área <AED> (juntamente com o ficheiro fonte `SortedList.java`, pode também lá encontrar a respetiva documentação javadoc em formato PDF).
6. É proibido alterarem o que quer que seja da interface `SortedList`. Nos métodos que não conseguirem implementar, devem simplesmente defini-los (na `LinkedList`) com a linha de código: `throw new UnsupportedOperationException("Método não implementado!");`
7. Nas várias classes desenvolvidas (excluindo a `LinkedList`), o código deve incluir os comentários javadoc adequados à posterior produção da documentação da aplicação.
8. O trabalho deverá ser realizado exclusivamente com recurso às técnicas, estruturas e ferramentas abordadas nas aulas.

Considerações gerais

1. O trabalho deverá ser realizado por grupos de 2. Trabalhos individuais terão uma **penalização de 2 valores**.
2. Apenas serão aceites para avaliação trabalhos cuja implementação não apresente qualquer erro de compilação e com um conjunto mínimo de funcionalidades operacionais.
3. É expressamente proibida a cópia integral ou parcial de código de outras fontes que não a documentação disponibilizada pelo docente da unidade curricular.
4. O trabalho deverá ser entregue apenas por um dos elementos do grupo, dentro do prazo estabelecido, obrigatoriamente no portal de e-learning (em <http://virtual.ipb.pt/>, escolher <Trabalho Pratico> no separador <[Assignments](#)>, dentro da área de <AED>), não sendo, em nenhuma circunstância, aceite o envio por e-mail. **[penalização de 1 valor se submeterem ambos os elementos do grupo]**
5. Deverão ser submetidos dois ficheiros, em anexos separados **[penalização 0.5 se submeterem num só anexo]**:
 - **trab_pratico.zip** – pasta principal do projeto depois de compactado pelo próprio NetBeans. Para o efeito, no menu <File> do NetBeans, selecionar Export Project->To ZIP (confirmar que o arquivo fica com a extensão “.zip”). **[penalização de 0.5 se outra extensão]**
 - **autores.txt** – ficheiro de texto contendo unicamente o número mecanográfico e o nome dos dois autores do trabalho, um autor por linha e exatamente no seguinte formato: número espaço primeiro e último nome (sem quaisquer outros caracteres e com o número só contendo dígitos).
Exemplo: 54321 Ana Sousa
 12345 Rui Esteves**[O não cumprimento escrupuloso desta regra implica uma penalização de 0.5 valores]**
6. O trabalho apenas poderá ser submetido com um atraso máximo de 5 dias, implicando nesse caso a **subtração de um valor por cada dia de atraso**.
7. Não serão permitidas resubmissões (quando submeter, certifique-se de que se trata da versão final).
8. Os trabalhos com classificação igual ou superior a 16 valores implicam obrigatoriamente defesa oral. O docente poderá igualmente solicitar a defesa presencial de quaisquer outros trabalhos, em data a definir, devendo os alunos demonstrar capacidade para implementar, compreender e explicar o código desenvolvido.
9. Para esclarecimentos adicionais sobre o trabalho, usar o fórum de discussão criado na plataforma <http://virtual.ipb.pt/>: AED (25/2.4) > Chat Room > [Trabalho Pratico](#).
(Para que todos beneficiem dos esclarecimentos prestados, não serão tiradas dúvidas por email)

¹ Na classe AED deverão ser implementados todos os métodos definidos no diagrama, sendo apenas permitido, se necessário, acrescentar métodos auxiliares privados.